A high-speed CRC-32 Implementation on FPGA

Fangfei Cai
School of Information Science&Technology, Northwest University, Xi'an, Shaanxi, China
caifangfei1999@163.com

Yuchen Nie
Xi'an Microelectronics Technology Institute, Xi'an, Shaanxi, China
nnyychch@foxmail.com

Keran Zhang
Xi'an Microelectronics Technology Institute, Xi'an, Shaanxi, China
keran_zhang@outlook.com

Hangzai Luo*
School of Information Science&Technology, Northwest University, Xi'an, Shaanxi, China
*Corresponding author: hzluo@nwu.edu.cn

Yanyan Li
School of Information Science&Technology, Northwest University, Xi'an, Shaanxi, China
liyanyan@nwu.edu.cn

2024 4th International Conference on Neural Networks, Information and Communication (NNICE) | 979-8-3503-9437-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/NNICE61279.2024.10499172

***Abstract*—Cyclic Redundancy Check (CRC) is widely used for transmission error detection in various communication interfaces. As the transmission rate increases, accelerating CRC with lower resource consumption for high-speed interfaces becomes significant. This paper analyzes and implements a typical CRC algorithm (Stride-x) and designs a padding-zero strategy to support the input data length with multiples of byte. Besides, experiments are conducted to validate the proposed algorithm on Xilinx FPGA platforms. When stride is 1, the proposed algorithm outperforms a typical parallel CRC algorithm in throughput and resource consumption with various input bus widths (32/128/256 bits).**

Keywords-CRC; Lookup table; FPGA

I. INTRODUCTION

Cyclic Redundancy Check (CRC) is widely used to verify the correctness of communication in computer networks [1]. As the throughput of network increases, the performance of error check becomes more important.

Hardware-based CRC mainly includes serial and parallel CRC algorithms. The serial algorithm generally uses a linear feedback shift register (LFSR) to support input data with arbitrary length. However, its latency may not satisfy the requirement of networks such as the 5G network, Ethernet and controller area network (CAN). The parallel algorithm pre-computes the CRC result for each bit of the input data, and accelerates the CRC by merging operations such as shift and conditional subtraction [2]. The related works focus on the optimization and hardware implementation of parallel algorithms [3][4][5]. Besides, CRC implementations often apply the lookup table to store possible remainders of each CRC sub-step, which simplifies the calculation [6][7][9].

This paper analyzes the principle of a typical lookup table-based CRC method, the Stride-x algorithm [8][13]. After adopting the padding-zero mechanism, the Stride-x algorithm can support input data length with multiples of byte. Besides, this paper implements the Stride-x algorithm for CRC-32 on FPGA. The implementation achieves the highest throughput and the lowest LUT resource consumption when x is equal to 1. Stride-1 algorithm also achieves higher throughput per LUT compared to a typical parallel CRC algorithms [14]. The proposed implementations can be used as IP in the design of high-speed interfaces.

II. ARCHITECTURE DESIGN

The basic principle of CRC is modulo-2 algorithm, which performs long division on the input data using a given polynomial. The remainder of the division is the CRC result. The CRC-k algorithm generally refers to a k bit CRC polar that has a specified value. This article mainly uses the idea of Stride-x to reduce logic latency, save resource consumption, improve bandwidth, and support input data with any multiple of bytes.

A. Stride-x Algorithm

Stride-x is a lookup table-based CRC algorithm [8]. It effectively improves computing speed and hardware resource utilization by partitioning the lookup table index to x-bit units. When the input data packet is large, it needs to enter the Stride-x module stage by stage. Assuming that the input data is I (n bits) for the current stage, I needs to perform the "bitwise XOR" operation with the remainder R of the previous stage before entering the Stride-x module. After the operation, the divisor $G = \{ g_{n-1}, g_{n-2}, ..., g_0 \}$ can be divided into multiple data d (x bits) according to the "bit slicing" and "bit replacement" principles of modulo-2 algorithm [8]. The step size x and the number of divisions m satisfy: $m = \frac{n}{x}$. d is represented as:

$$d_{m-1} = \{ g_{n-1}, g_{n-2}, ..., g_{n-x} \}, \quad (1)$$

$$d_{m-2} = \{ g_{n-x-1}, g_{n-x-2}, ..., g_{n-2x} \}, \quad (2)$$

$$\ldots,$$

$$d_0 = \{ g_{x-1}, g_{x-2}, ..., g_0 \} \quad (3)$$

After padding zeros, the data width of d is n bits. There are m data in total, which forms D as follows:

$$D_{m-1} = \{ d_{m-1}, 0, ..., 0 \}, \quad (4)$$

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on October 15,2026 at 02:05:23 UTC from IEEE Xplore. Restrictions apply.

$$D_{m-2} = \{ 0 , d_{m-2}, ..., 0\} , \quad (5)$$

…,

$$D_0 = \{ 0, 0, ..., d_0 \} \quad (6)$$

Let the polynomial used for long division be *P*. The remainder *R* in the current stage satisfies:

$$\begin{aligned} R &= G \bmod P \\ &= \left(D_{m-1} \oplus D_{m-2} \oplus ... \oplus D_0\right) \bmod P \\ &= \left(D_{m-1} \bmod P\right) \oplus \left(D_{m-2} \bmod P\right) \oplus ... \oplus \left(D_0 \bmod P\right) \\ &= R_{m-1} \oplus R_{m-2} \oplus ... \oplus R_0 \end{aligned} \quad (7)$$

In Stride-x algorithm, the calculation of the remainder *Ri* (0<i<m-1) can be replaced with *m* lookup tables. Then the final CRC result *R* for the *n* bits input data is calculated using multiple steps of pairwise XOR (XOR tree) on *Ri*. The construction of the lookup tables in Stride-x algorithm is shown in Algorithm 1. Table$_{m-1}$ is the *m-1* independent lookup tables generated by stride-1 algorithm. The maximum length of N_0 (*x* bit) is 2^x-1.

Algorithm 1:The construction of the lookup table

Input: N_0

Output: table$_{m-1}$, table$_{m-2}$, …, table$_0$

Initialize P to 0x4C11DB7

```
1:      for j = 0 to N0 do
2:          r = j % P;
3:          table1[j] = r;
4:      end for
5:      for i = 1 to m-1 do
6:          for j = 0 to N0 do
7:              r = tablei[j];
8:              r = (r<<x) XOR tablei[r>>(32-x)];
9:              tablei+1[j] = r;
10:         end for
11:     end for
```

Algorithm 1 first generates the lookup table with index d_{m-1} (lines 1-4), then performs shifts and XORs to obtain the remaining lookup tables with indices d_{m-2}, d_{m-3}, …, d_0. The value of P is calculated using CRC-32-IEEE 802.3: P(*k*) = $X^{32}+X^{26}+X^{23}+X^{22}+X^{16}+X^{12}+X^{11}+X^{10}+X^8+X^7+X^5+X^4+X^2+X^1+1$. The construction algorithm is implemented in advance through software, and the generated lookup table is implemented in FPGA. This function expression belongs to the *m-1*-th lookup table:

$$table_{m-1}\left(d_{m-1}\right)$$

The Stride-x algorithm uses the lookup table to quickly obtain the CRC result in each stage. Since *x* is fixed, the delay of the Stride-x algorithm mainly depends on the critical path of the XOR operation. The use of XOR tree may result in a latency comparable to that of parallel algorithm.

B. Padding-zero Mechanism

Traditional CPUs generally use 32 bit or 64 bit data types, while the transmission data unit for communication devices (such as serial communication) is less than 32 bit. In Stride-x algorithm, the bus should append zeros to the lower bits of the input (to *n* bit) before the final stage. Therefore, this paper proposes a mechanism based on XOR gate, which solves the padding-zero problem and enables the Stride-x algorithm to support input data widths of any multiples of byte. The padding-zero mechanism consists of pre-XOR and post-XOR modules, and the logical architecture is shown in Figure 1.

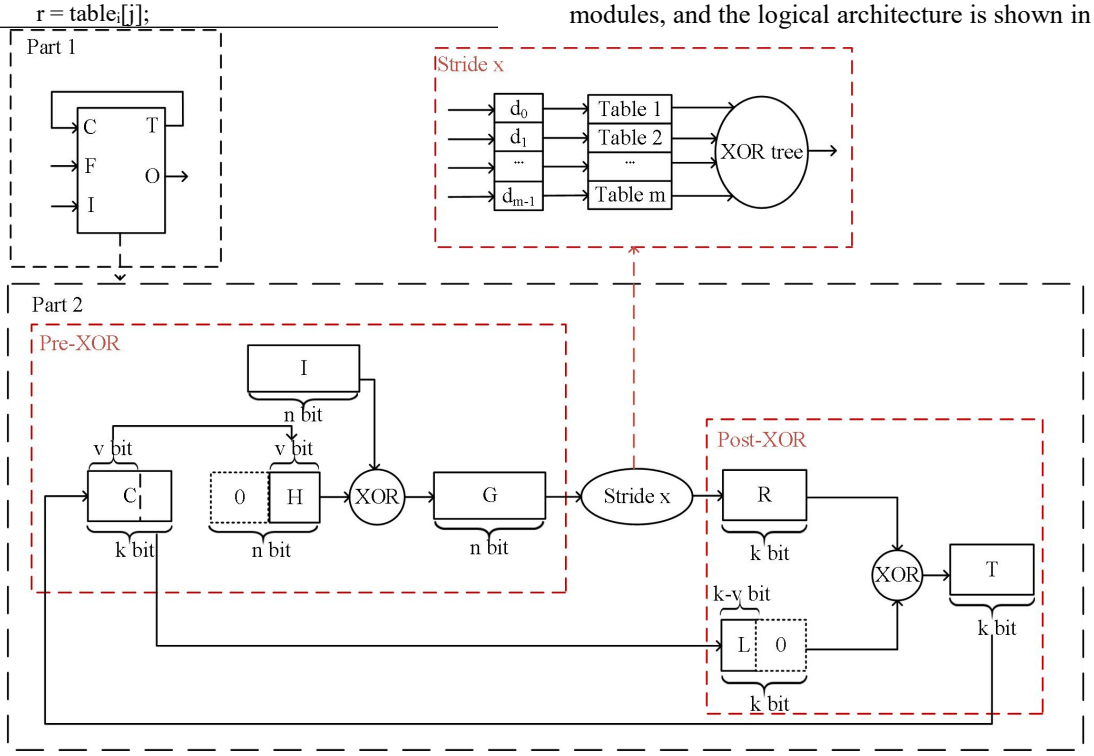


Figure 1. Logic diagram of Stride-x algorithm and padding-zero modules

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on October 15,2026 at 02:05:23 UTC from IEEE Xplore. Restrictions apply.

In Figure 1, part 1 is the top module, T (k bit) is the CRC result of the current stage; C (k bit) represents T as the input of the next stage (which is initialized to all 1s at the first stage); I (n bit) is another input for each stage; F (1 bit) indicates whether the current stage is the final stage; *O* (*k* bit) is the final CRC result after all stages end.

Part 2 consists of the padding-zero module and the Stride-x module, while the padding-zero module includes pre-XOR module and post-XOR module. The pre-XOR divides *C* into two parts. The higher *v* bits are left-padded with zeros to get *H* (*n* bits), and the lower (*k*-*v*) bits are right-padded with zeros to get L (*k* bits). The dividend *G* (*n* bits) comes from the XOR operation of *H* and *I*. In each stage, the Stride-x module calculates the CRC remainder *R* (*k* bits) for *G*. Then the post-XOR performs XOR operation for *L* and *R* and gets the CRC result *T*. The result *T* in the final stage is the output data *O*.

III. Experimental Results

We conduct experiment for CRC-32, which is a common CRC polar. The experiments mainly study the impact of the stride value (*x*) and the input data width (*n*) on throughput and resource consumption.

Different works have focused on the implementation of CRC on FPGA platforms [10] [11] [12] in recent years. This paper implements the stride-x algorithm on Xilinx Virtex, Kintex, and Zynq series. The synthesis is performed using Vivado 2021 software. The clock frequency is 100MHz. We obtain the result of resource consumption, maximum clock frequency and transmission throughput.

Firstly, we conducts experiments for the Stride-x module on Virtex (xc7vx690t) with different values for *x* and *n*. The results are shown in Figure 2 and Figure 3. The LUT consumption and throughput increase linearly with *n*, and vary slightly with *x*. When *x*=1, the implementation achieves the maximum throughput and minimum LUT consumption. The same conclusion applies to Kintex and Zynq.

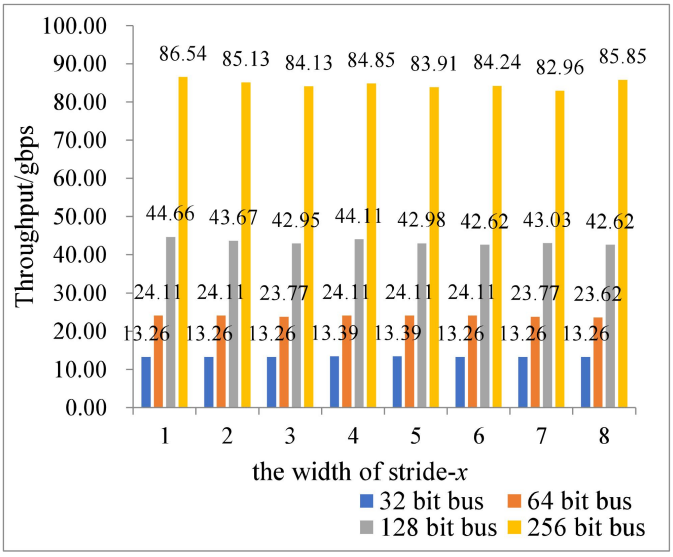


Figure 2. The throughput of Stride-x

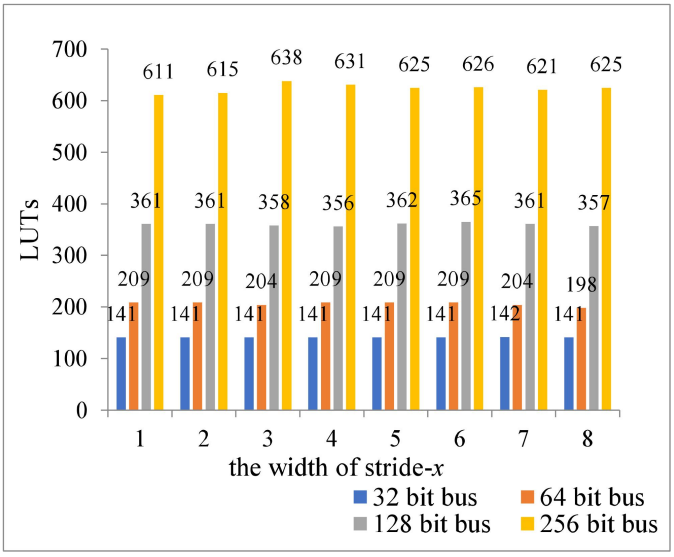


Figure 3. The LUTs of Stride-x

Table 1. The performance under different bus widths on Virtex/Kinex

	Stride-1 with padding-zero				Parallel CRC with padding-zero			
	32 bit	64 bit	128 bit	256 bit	32 bit	64 bit	128 bit	256 bit
LUTs	260	406	508	835	273	320	506	798
FFs	97	97	97	97	97	97	97	97
WNS(ns)	7.109	6.901	6.408	6.302	6.944	6.721	6.383	5.988
WHS(ns)	0.355	0.402	0.496	0.402	0.317	0.312	0.314	0.317
Fmax(Mhz)	345.90	322.68	278.40	270.42	327.23	304.97	276.47	249.25
Throughput(gbps)	11.07	20.65	35.63	69.23	10.47	19.52	35.39	63.81
Throughput/ LUTs	0.043	0.051	0.070	0.083	0.038	0.061	0.070	0.080

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on October 15,2026 at 02:05:23 UTC from IEEE Xplore. Restrictions apply.

Table 2. The performance under different bus widths on Zynq

	Stride-1 with padding-zero				Parallel CRC with padding-zero			
	32 bit	64 bit	128 bit	256 bit	32 bit	64 bit	128 bit	256 bit
LUTs	260	406	508	835	273	320	506	798
FFs	97	97	97	97	97	97	97	97
WNS(ns)	6.349	6.021	5.405	5.156	6.133	5.908	5.247	4.849
WHS(ns)	0.55	0.609	0.711	0.609	0.483	0.478	0.48	0.061
Fmax(Mhz)	273.90	251.32	217.63	206.44	258.60	244.38	210.39	194.14
Throughput(gbps)	8.76	16.08	27.86	52.85	8.28	15.64	26.93	49.70
Throughput/ LUTs	0.034	0.040	0.055	0.063	0.030	0.049	0.053	0.062

Secondly, we conduct experiments to compare the Stride-1 with a typical parallel CRC algorithm [14]. The results are shown in Table 1 and Table 2. When the input data width are 32 bit/128 bit/256 bit, the stride-1 achieves a higher throughput than the parallel CRC algorithm. After adding padding-zero, the critical path of the Stride-x algorithm becomes longer, resulting in a slight decrease in throughput.

IV. Conclusions

This paper analyzes the Stride-x method, designs the logic modules and adds the padding-zero mechanism. In the experiments for CRC-32 on FPGA, the Stride-x algorithm has good scalability for different bus widths. When x=1, the LUT resource utilization is the lowest and the throughput is the highest. In addition, the FPGA performance of the stride-1 algorithm is better than that of the typical parallel CRC algorithm.

References

[1] Peterson, W. W. and Brown, D. T. (January 1961). "Cyclic Codes for Error Detection". Proceedings of the IRE 49 (1): 228–235. doi:10.1109/JRPROC.1961.287814.

[2] T. . -B. Pei and C. Zukowski, "High-speed parallel CRC circuits in VLSI," in IEEE Transactions on Communications, vol. 40, no. 4, pp. 653-657, April 1992, doi: 10.1109/26.141415.

[3] N. N. Qaqos, "Optimized FPGA Implementation of the CRC Using Parallel Pipelining Architecture," 2019 International Conference on Advanced Science and Engineering (ICOASE), 2019, pp. 46-51, doi: 10.1109/ICOASE.2019.8723800.

[4] C. E. Kennedy and M. Mozaffari-Kermani, "Generalized parallel CRC computation on FPGA," 2015 IEEE 28th Canadian Conference on Electrical and Computer Engineering (CCECE), 2015, pp. 107-113, doi: 10.1109/CCECE.2015.7129169.

[5] M. Walma, "Pipelined Cyclic Redundancy Check (CRC) Calculation," 2007 16th International Conference on Computer Communications and Networks, Honolulu, HI, USA, 2007, pp. 365-370, doi: 10.1109/ICCCN.2007.4317846.

[6] [1]Sarwate, D. V .Computation of cyclic redundancy checks via table look-up[J].Communications of the ACM, 1988, 31(8): 1008-1013. DOI:10.1145/63030.63037.

[7] Y. Huo, X. Li, W. Wang and D. Liu, "High performance table-based architecture for parallel CRC calculation," The 21st IEEE International Workshop on Local and Metropolitan Area Networks, Beijing, China, 2015, pp. 1-6, doi: 10.1109/LANMAN.2015.7114717.

[8] F M. E. Kounavis and F. L. Berry, "Novel Table Lookup-Based Algorithms for High-Performance CRC Generation," in IEEE Transactions on Computers, vol. 57, no. 11, pp. 1550-1560, Nov. 2008, doi: 10.1109/TC.2008.85.

[9] Dong X , He Y .CRC Algorithm for Embedded System Based on Table Lookup Method[J].Microprocessors and Microsystems, 2020, 74:103049.DOI:10.1016/j.micpro.2020.103049

[10] Q. Clark Shen, J. C. Vega and P. Chow, "Parallel CRC On An FPGA At Terabit Speeds," 2022 International Conference on Field-Programmable Technology (ICFPT), Hong Kong, 2022, pp. 1-6, doi: 10.1109/ICFPT56656.2022.9974233.

[11] J. Mitra and T. K. Nayak, "Reconfigurable Concurrent VLSI (FPGA) Design Architecture of CRC-32 for High-Speed Data Communication," 2015 IEEE International Symposium on Nanoelectronic and Information Systems, Indore, India, 2015, pp. 112-117, doi: 10.1109/iNIS.2015.66.

[12] J. Cabal, L. Kekely and J. Kořenek, "High-Speed Computation of CRC Codes for FPGAs," 2018 International Conference on Field-Programmable Technology (FPT), Naha, Japan, 2018, pp. 234-237, doi: 10.1109/FPT.2018.00042.

[13] H. Liu, Z. Qiu, W. Pan, J. Li, L. Zheng and Y. Gao, "Low-Cost and Programmable CRC Implementation Based on FPGA," in IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 68, no. 1, pp. 211-215, Jan. 2021, doi: 10.1109/TCSII.2020.3008932.

[14] Stavinov E .A Practical Parallel CRC Generation Method[J]. [2023-12-21]. Available:https://www.researchgate.net/publication/265523608_A_Practical_Parallel_CRC_Generation_Method

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on October 15,2026 at 02:05:23 UTC from IEEE Xplore. Restrictions apply.